



Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251229
Nama Lengkap	Ignatius Harya Nugraha
Minggu ke / Materi	07 / Percabangan & Perulangan Kompleks

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Struktur Percabangan

Struktur percabangan paling dasar dalam if. Struktur ini digunakan ketika program perlu menjalankan suatu perintah hanya jika kondisi tertentu terpenuhi. Jika kondisi bernilai True, maka blok kode di bawahnya akan dieksekusi. Jika bernilai False, blok tersebut akan dilewati tanpa menimbulkan kesalahan.

Penggunaan if menjadi sangat efektif ketika hanya ada satu kondisi utama yang ingin diuji. Namun dalam banyak kasus, program memerlukan dua kemungkinan hasil. Dalam situasi tersebut digunakan struktur if-else. Struktur ini memaksa program untuk memilih satu dari dua jalur yang tersedia. Ketika kondisi terpenuhi, jalur pertama dijalankan; ketika tidak terpenuhi, jalur alternatif akan dijalankan.

Namun apabila terdapat lebih dari dua kemungkinan, digunakan struktur if-elif-else. Struktur ini memungkinkan evaluasi beberapa kondisi secara berurutan. Program akan membaca kondisi dari atas ke bawah dan mengeksekusi blok kode pertama yang bernilai True. Karena itu kita perlu untuk Menyusun logika urutannya agar program dapat berjalan sesuai dengan keinginan kita.

Contoh penulisan struktur if, if-else, if-elif-else:

```
nilai = 78

if nilai >= 60:
    print("Selamat, kamu lulus!")

if nilai >= 60:
    print("Lulus")
else:
    print("Tidak Lulus")

if nilai >= 85:
    print("Grade A")
elif nilai >= 75:
    print("Grade B")
elif nilai >= 65:
    print("Grade C")
else:
    print("Grade D/E")
```

Operator Logika dan Kompleksitas Kondisi

Dalam penerapan suatu logika dalam pemrograman satu kondisi sering kali tidak cukup untuk mendapatkan hasil yang akurat. Oleh karena itu, terdapat operator logika seperti and, or, dan not. Operator ini memungkinkan penggabungan beberapa kondisi menjadi satu ekspresi yang lebih kompleks.

Operator and berarti semua kondisi bernilai True agar hasil akhirnya True. Operator ini sering digunakan dalam situasi yang membutuhkan beberapa persyaratan sekaligus. Sebaliknya, operator or hanya membutuhkan satu kondisi bernilai True agar keseluruhan ekspresi bernilai True. Operator ini cocok digunakan dalam situasi yang memiliki beberapa alternatif pemenuhan syarat. Sementara itu, operator not digunakan untuk membalik nilai kebenaran suatu kondisi.

Contoh penggunaan operator logika pada python:

```
umur = 20
punya_ktp = True
nilai_ujian = 72

if umur >= 17 and punya_ktp:
    print("Bisa mendaftar layanan")

if nilai_ujian >= 75 or nilai_ujian >= 60:
    print("Boleh mengikuti remedial")

if not punya_ktp:
    print("Harap buat KTP terlebih dahulu")
```

Penggunaan operator logika pada pemrograman bukan hanya soal memeriksa satu kondisi tunggal, melainkan sering kali melibatkan kombinasi berbagai hal yang saling berkaitan. Kemampuan menyusun kondisi kompleks dengan operator logika menjadi indikator pemahaman logika yang lebih matang.

If-Elif-Else pada Ternary If

Struktur if-elif-else merupakan perluasan dari if-else yang memungkinkan pengecekan beberapa kondisi secara berurutan. Kata kunci elif adalah singkatan dari "else if", artinya kondisi ini hanya akan diperiksa jika kondisi-kondisi sebelumnya bernilai False. Dengan demikian, hanya satu blok kode yang akan dieksekusi dalam satu kali jalannya program.

Ternary if, atau disebut juga conditional expression, adalah cara singkat untuk menuliskan if-else dalam satu baris. Sintaksnya adalah: nilai_jika_true if kondisi else nilai_jika_false. Ternary if berguna ketika kita ingin menetapkan nilai suatu variabel berdasarkan kondisi tertentu secara ringkas. Namun untuk kondisi yang kompleks, tetap disarankan menggunakan if-else biasa agar kode lebih mudah dibaca dan dipahami.

Contoh penerapan Ternary If dalam python:

```
nilai = 68
status = "A" if nilai >= 80 else "B" if nilai >= 70 else "C"
print(f"Nilai dalam huruf: {status}")
```

Nested If dan Struktur Keputusan Bertingkat

Nested if (percabangan bersarang) adalah penggunaan struktur if di dalam fungsi if lainnya. Struktur ini digunakan ketika keputusan yang perlu diambil terdiri dari beberapa tahap yang saling bergantung satu sama lain. Kondisi di dalam hanya akan diperiksa apabila kondisi di luarnya sudah terpenuhi terlebih dahulu. Hal ini memberikan ruang untuk program memodelkan proses pengambilan keputusan yang lebih mendetail dan hierarkis.

Meskipun nested if sangat berguna, penggunaannya yang terlalu dalam dapat membuat kode menjadi sulit dibaca dan dipahami. Ini sering disebut sebagai "pyramid of doom" karena bentuk indentasi kode yang semakin menjorok ke kanan. Umumnya disarankan untuk tidak membuat percabangan lebih dari tiga tingkat. Jika diperlukan, pertimbangkan untuk memecah logika menjadi beberapa fungsi terpisah agar kode tetap bersih dan mudah dipelihara.

Berikut adalah contoh kode dalam penerapan nested if dalam system pendaftaran beasiswa berdasarkan keaktifan, penghasilan orang tua, dan IPK:

```
ipk = float(input("Masukkan IPK kamu: "))
penghasilan_ortu = int(input("Penghasilan orang tua (Rp/bulan): "))
aktif_organisasi = input("Aktif di organisasi? (ya/tidak): ")

if ipk >= 3.0:
    print("IPK memenuhi syarat.")
    if penghasilan_ortu <= 3000000:
        print("Penghasilan orang tua memenuhi syarat.")
        if aktif_organisasi == "ya":
            print("Selamat! Kamu lolos seleksi beasiswa penuh.")
        else:
            print("Kamu lolos beasiswa parsial (tidak aktif organisasi).")
    else:
        print("Maaf, penghasilan orang tua melebihi batas.")
else:
    print("Maaf, IPK kamu belum memenuhi syarat minimum 3.0.")
```

Pada contoh di atas, program pertama-tama memeriksa apakah IPK mahasiswa memenuhi syarat. Jika ya, baru diperiksa kondisi penghasilan orang tua. Jika itu pun terpenuhi, barulah status keaktifan organisasi diperiksa untuk menentukan jenis beasiswa yang didapatkan. Ini adalah contoh pengambilan keputusan bertingkat yang mencerminkan alur seleksi nyata.

Error Handling menggunakan Try dan Except

Error handling adalah mekanisme dalam pemrograman yang digunakan untuk menangani kesalahan (error) yang terjadi saat program berjalan. Tanpa error handling, sebuah error sekecil apapun dapat menyebabkan program berhenti secara tiba-tiba dan tidak terkendali. Dalam sebuah program, error dibedakan menjadi dua jenis utama: Syntax Error dan Runtime Error (Exception).

Syntax Error terjadi saat kode tidak ditulis sesuai aturan bahasa Program, sehingga program tidak bisa berjalan sama sekali. Sementara Exception terjadi saat program sudah berjalan tetapi menemui situasi yang tidak bisa ditangani, seperti membagi angka dengan nol, mengakses index di luar batas list, atau memasukkan tipe data yang tidak sesuai. Beberapa jenis exception yang sering ditemui:

- ValueError – terjadi saat fungsi menerima nilai yang tidak sesuai, misalnya memasukkan huruf saat program meminta angka.
- ZeroDivisionError – terjadi saat terjadi pembagian dengan angka nol.
- TypeError – terjadi saat operasi diterapkan pada tipe data yang tidak sesuai.
- IndexError – terjadi saat mengakses indeks list yang berada di luar batas.
- FileNotFoundError – terjadi saat file yang ingin dibuka tidak ditemukan di sistem.

Struktur try-except adalah cara Python menangani error secara terstruktur dan terkontrol. Blok try berisi kode yang berpotensi menimbulkan error. Jika error terjadi di dalam blok try, program tidak langsung berhenti, melainkan akan melompat ke blok except yang sesuai untuk menangani error tersebut. Ini memungkinkan program untuk tetap berjalan dan memberikan respons yang informatif kepada pengguna, bukan sekadar pesan error yang membingungkan.

Struktur lengkap try-except juga dapat digabung dengan if else. Perintah else akan dijalankan hanya jika tidak ada error yang terjadi pada blok try, berguna untuk kode lanjutan yang hanya relevan jika proses berhasil. Blok finally akan selalu dijalankan baik ada error maupun tidak. Blok finally umumnya digunakan untuk membersihkan resource, seperti menutup koneksi database atau file yang sudah dibuka, agar tidak terjadi kebocoran resource (resource leak).

Definisi Perulangan

Perulangan(loop) merupakan salah satu struktur kontrol utama dalam pemrograman. Dalam Python, jalannya program dapat diatur secara sekuensial, percabangan, perulangan, maupun kombinasi dari ketiganya. Secara umum, perulangan digunakan ketika program perlu:

- Melakukan suatu hal yang sama beberapa kali secara berulang-ulang.
- Melakukan suatu hal secara bertahap, di mana setiap tahap sebenarnya memiliki langkah yang sama.
- Mengakses sekumpulan data dalam suatu struktur data, misalnya List, Tuple, Queue, Stack, dan beberapa struktur data lainnya

Dalam penerapannya perulangan pada kehidupan sehari-hari dapat kita lihat pada saat seorang guru yang harus mengabsen 30 siswa di kelas. Proses yang dilakukan adalah proses yang terus

berulang yaitu memanggil nama dan menunggu respons untuk setiap siswa. Inilah gambaran dari perulangan dalam pemrograman.

Pada Python, perulangan dapat dilakukan dengan menggunakan for, while, maupun dengan cara rekursif. Pada laporan praktikum pertemuan ini, akan dibahas mengenai perulangan for dan while beserta kontrol perulangan menggunakan break dan continue.

Perulangan For

Perulangan for pada Python biasanya digunakan pada kondisi di mana jumlah perulangan sudah diketahui sejak awal, atau perulangan terjadi karena operasi yang sama pada suatu rentang data atau rentang nilai tertentu. Perulangan for pada rentang tertentu lebih mudah dilakukan dengan menggunakan fungsi range().

Fungsi Range

Fungsi range() pada Python memiliki beberapa bentuk penggunaan, yaitu:

- **range(stop)** Ini berarti range akan menghasilkan rentang dari 0 sampai stop-1. Contoh: range(6) menghasilkan 0, 1, 2, 3, 4, 5.
- **range(start, stop)** Dengan tambahan start maka akan menghasilkan rentang dari start sampai stop-1 dengan langkah default 1.
- **range(start, stop, step)** Tambahan step dibelakang akan menghasilkan rentang dari start sampai stop-1 dengan langkah sejumlah step (bisa positif maupun negatif).

Contoh Penggunaan

Berikut adalah contoh program untuk menampilkan bilangan dari 1 sampai 100:

```
for i in range(1, 101):  
    print(i)
```

Pada program di atas, variabel i berfungsi sebagai counter yang nilainya naik secara berurutan sesuai nilai yang dihasilkan fungsi range(1, 101), yaitu mulai dari 1 sampai 100. Jika counter tidak diperlukan, dapat digunakan underscore (_) sebagai pengganti variabel counter:

```
for i in range(100):
```

```
print('Hello World')
```

Program tersebut akan mencetak teks 'Hello World' sebanyak 100 kali tanpa membutuhkan nilai counter.

Step Negatif

Fungsi `range()` juga dapat menerima step negatif, yang berguna ketika diperlukan perulangan dari nilai besar ke nilai kecil. Berikut contoh menampilkan bilangan genap dari 100 sampai 2:

```
for i in range(100, 1, -2):  
    print(i)
```

Program tersebut menampilkan bilangan genap mulai dari 100, 98, 96, 94, ..., sampai 2

Perulangan While

Berbeda dengan `for`, perulangan `while` umumnya digunakan pada kondisi di mana jumlah perulangan belum diketahui sebelumnya. Perulangan `while` akan terus berjalan selama kondisi yang ditetapkan masih bernilai `True`. Struktur umum perulangan `while` adalah sebagai berikut:

```
kondisi=True  
while kondisi== True:  
    print("Hello World")  
    kondisi=False
```

Tiga hal penting yang harus ada dalam perulangan `while` adalah: nilai awal (untuk memulai perulangan), kondisi akhir (untuk menentukan kapan perulangan berhenti), dan langkah/step (agar iterasi dapat berjalan dari nilai awal hingga mencapai kondisi akhir).

Contoh Penggunaan while

Berikut adalah contoh perulangan `while` untuk memastikan input yang dimasukkan oleh pengguna adalah bilangan genap:

```
bilangan = 0  
genap = False  
while genap == False:
```

```
bilangan = int(input('Masukkan bilangan genap: '))
if bilangan % 2 == 0:
    genap = True
print(bilangan, 'yang anda masukkan adalah bilangan genap')
```

Program di atas terus meminta input hingga pengguna memasukkan bilangan genap. Kasus ini sangat sesuai menggunakan while karena tidak diketahui berapa kali pengguna akan memasukkan nilai yang salah sebelum akhirnya memberikan bilangan genap.

BAGIAN 2: LATIHAN MANDIRI (60%)

Latihan 6.1

Buatlah program untuk mencari bilangan prima terdekat dari suatu bilangan yang diinputkan oleh pengguna (n), dengan syarat bilangan prima tersebut $< n$. Contoh: $n=12 \rightarrow 11$ | $n=21 \rightarrow 19$

```
def is_prima(num):
    if num < 2:
        return False
    for i in range(2, int(num ** 0.5) + 1):
        if num % i == 0:
            return False
    return True

def prima_terdekat(n):
    kandidat = n - 1
    while kandidat >= 2:
        if is_prima(kandidat):
            return kandidat
        kandidat -= 1
    return None

n = int(input("Masukkan nilai n: "))

hasil = prima_terdekat(n)

if hasil is not None:
    print(f"Prima terdekat < {n} adalah {hasil}")
else:
    print(f"Tidak ada bilangan prima yang kurang dari {n}")
```

1. Fungsi is_prima(num):

- Memeriksa apakah input bilangan adalah bilangan prima.
- Jika $num < 2$, langsung return False (bukan prima).
- Perulangan dilakukan dari 2 hingga akar kuadrat dari num.
Trik ini menghemat waktu karena faktor selalu berpasangan di bawah dan di atas akar kuadrat.
- Jika ditemukan pembagi ($num \% i == 0$), bilangan bukan prima $\rightarrow$ False.
- Jika tidak ada pembagi ditemukan $\rightarrow$ True (prima).

2. Fungsi `prima_terdekat(n)`:

- Memulai pencarian dari $n-1$ (satu di bawah n).
- Menggunakan perulangan `while` menurun hingga angka 2.
- Setiap angka diperiksa dengan `is_prima()`. Jika prima, langsung `return`.
- Jika tidak ada prima yang ditemukan, `return None`.

3. Blok IF–ELSE di main program:

- Jika hasil tidak `None` maka cetak bilangan prima yang ditemukan.
- Jika `None` kembalikan nilai beri tahu pengguna tidak ada $\text{prima} < n$.

Latihan 6.2

Buatlah program untuk menampilkan deret seperti di bawah ini. n diinputkan secara dinamis

contoh: $n = 6$

720 6 5 4 3 2 1

120 5 4 3 2 1

24 4 3 2 1

6 3 2 1

2 2 1

1 1

```
def faktorial(num):  
    hasil = 1  
    for i in range(1, num + 1):  
        hasil *= i  
    return hasil  
  
n = int(input("Masukkan nilai n: "))  
  
for baris in range(n, 0, -1):  
    fak = faktorial(baris)  
  
    deret = " ".join(str(i) for i in range(baris, 0, -1))  
  
    print(f"{fak} {deret}")
```

1. Fungsi faktorial(num):

- Menghitung nilai faktorial secara iteratif (bukan rekursif).
- Dimulai dari hasil = 1, kemudian dikalikan setiap bilangan dari 1 hingga num.
- Contoh: $\text{faktorial}(5) = 1 \times 2 \times 3 \times 4 \times 5 = 120$.

2. Perulangan utama (for baris in range(n, 0, -1)):

- Baris pertama dimulai dari n, kemudian turun hingga 1.
- Setiap iterasi mewakili satu baris pada output.

3. Konstruksi setiap baris:

- fak = faktorial(baris): hitung faktorial untuk baris saat ini.
- deret = " ".join(...): gabungkan angka dari baris...1 menjadi string.
- Contoh baris ke-1 (baris=6): fak=720, deret="6 5 4 3 2 1"
→ Output: "720 6 5 4 3 2 1"

4. Pola yang terbentuk:

- Kolom pertama selalu faktorial dari bilangan terbesar di baris tersebut.
- Kolom berikutnya adalah deret menurun dari bilangan baris hingga 1.
- Setiap baris baru, panjang deret berkurang satu.

Latihan 6.3

Buatlah program untuk menampilkan deret seperti di bawah ini. n diinputkan secara dinamis

contoh: tinggi = 5, lebar = 4

1 2 3 4

5 6 7 8

9 10 11 12

13 14 15 16

17 18 19 20

```
tinggi = int(input("Masukkan tinggi: "))
lebar = int(input("Masukkan lebar : "))

angka = 1
for baris in range(tinggi):
    isi_baris = []
    for kolom in range(lebar):
        isi_baris.append(str(angka))
        angka += 1
    print("\t".join(isi_baris))
```

1. Input tinggi dan lebar:

- tinggi adalah jumlah baris yang akan dicetak.
- lebar adalah jumlah kolom (angka per baris).

2. Variabel angka = 1:

- Counter tunggal yang terus bertambah 1 setiap elemen.
- Tidak di-reset per baris, sehingga angka mengalir dari baris satu ke baris berikutnya secara berurutan.

3. Loop bersarang (nested loop):

- Loop luar (for baris): mengulang sebanyak tinggi kali iterasi = satu baris pada output.
- Loop dalam (for kolom): mengulang sebanyak lebar kali iterasi = satu angka pada baris tersebut.
- Setiap angka ditambahkan ke list isi_baris, lalu angka dinaikkan.

4. Output:

- "\t".join(isi_baris) berfungsi menggabungkan semua angka dalam satu baris dengan pemisah TAB agar kolom terlihat rapi.

- print() dipanggil di luar loop dalam agar seluruh kolom dicetak dalam satu baris sekaligus.

Link Repository Github : <https://github.com/Haigry/PrakAIPro-71251229>